

BUILD GUIDE

How linqsafe is built, and how to build one yourself

A step-by-step walkthrough of the stack behind linqsafe, a link-in-bio platform with themes, templates, analytics, paid add-on features bought through Paystack, and a founder dashboard. Follow it in order to recreate a project like this from an empty folder.

Contents

1. What you're building
2. The stack, in plain words
3. What to install and sign up for
4. Step-by-step build (14 steps)
5. Going live: local, dev and prod
6. Launch checklist

1. What you're building

A website where anyone can sign up, pick a username, and get a page at `linqsafe.com/username` that lists their links. Owners manage the page from a dashboard; visitors see a fast, good-looking profile.

Part	What it does
Public profile	Photo, bio, social icons, links, WhatsApp button. 7 templates, 5 themes.
Dashboard	Onboarding wizard, add/reorder links, live phone preview, profile and photo, analytics.
Paid features	Unlimited links, premium templates, founder's note, testimonials, QR code, each bought on its own for 1, 3, 6 or 12 months in naira via Paystack. Prices set by the founder.
Founder dashboard	Whole-site stats on an <code>admin.</code> subdomain, only for the owner's email.

2. The stack, in plain words

Piece	Tool	What it's for
Language	JavaScript (Node.js 22)	One language for the website and the server.
Website (frontend)	React 18 + Vite	React builds the pages from components; Vite runs it while you work and bundles it for launch.
Pages & URLs	React Router	Decides which page shows for <code>/login</code> , <code>/admin</code> , <code>/username</code> .
Styling	Tailwind CSS	Style with short class names; colours live in one file as variables.
Animation	Framer Motion, GSAP, three.js	Framer for hovers, reveals and transitions; GSAP for scroll effects; three.js for the 3D tree.
Icons	lucide-react	Clean line icons.
Server (backend)	Express	Answers requests like "save this link" or "log me in" under <code>/api</code> .
Database	MySQL 8	Stores users, links, analytics events and payments.
Passwords & login	bcrypt + JWT in an httpOnly cookie	Passwords are scrambled before saving; the login lives in a cookie page scripts can't read.
Email	Resend	Sends "confirm your email" and "reset password" messages.
Payments	Paystack	Takes naira payments (card, transfer, USSD) and tells the server when someone paid.
Hosting	Vercel	Puts the site online, runs the server as a "serverless function", handles domains.
Hosted database	PlanetScale, Aiven or TiDB Cloud	MySQL in the cloud for dev and prod.
Containers (optional)	Docker + Docker Compose	Packs the app into one image for Docker hosts, or runs app + MySQL together. Not needed for local dev or Vercel.

3. What to install and sign up for

- **On your computer:** Node.js 22 (via nvm), MySQL 8 (`brew install mysql` on Mac), Git, VS Code. Docker Desktop only if you want the optional container setup.
- **Accounts:** GitHub (code), Vercel (hosting), a hosted MySQL provider, Resend (email), Paystack (payments), a domain name.

Tip: each step below ends with a **prompt** you can give an AI coding assistant (like Claude Code) to do that step for you. Run them one at a time and check the result before moving on.

4. Step-by-step build

Step 1: Create the project

```
npm create vite@latest linqsafe -- --template react
cd linqsafe && npm install
npm install react-router-dom framer-motion lucide-react
```

You now have a React site at `localhost:5173`.

Prompt: "Set up React Router with pages for Home, Login, Signup, Dashboard (/admin), Contact, Terms, Privacy and a public profile at /:username."

Step 2: Styling and design tokens

Install Tailwind, then define your colours once as CSS variables (`--background`, `--foreground`, `--accent` ...) in `src/styles.css`. Add Google Fonts. Build small reusable components: Button, Card, Input, Label.

```
npm install -D tailwindcss@3 postcss autoprefixer
npx tailwindcss init -p
```

Prompt: "Add Tailwind with shadcn-style Button, Card, Input and Label components. Palette: warm beige background, espresso text, maroon accent used at low opacity, plus rose, lilac, sand and dusty-blue tints. Fonts: Bricolage Grotesque for headings, Inter for body."

Step 3: Server and database

```
npm install express mysql2 dotenv helmet compression express-rate-limit express-async-errors
```

Create `server/app.js` (the routes), `server/db.js` (the connection) and `server/migrations.js`: a numbered list of database changes that run once each. Add a Vite proxy so `/api` calls reach the server in development.

Prompt: "Add an Express API under /api with a MySQL pool, versioned migrations recorded in a `schema_migrations` table, security headers with helmet, and rate limiting. Proxy /api from Vite to port 3001."

Step 4: Sign-up and log-in

```
npm install bcryptjs jsonwebtoken
```

Hash passwords with bcrypt. On log-in, put a signed JWT in an **httpOnly, SameSite=Lax** cookie (never in localStorage). Allow log-in with username or email. Record `last_login_at`.

Prompt: "Add register, login and logout endpoints using bcrypt and a JWT in an httpOnly cookie. Put Log in and Create account as two tabs on one card, each at its own URL."

Step 5: Links

Endpoints to add, edit, delete and reorder links. Detect the platform from the URL (instagram.com → Instagram) and show a coloured badge; ask the user to pick a type when it's unknown. On the server, reject a social badge that doesn't match the real domain.

Prompt: "Build a links CRUD with drag-to-reorder in the dashboard, platform detection from the URL with brand-coloured badges, and one-tap suggestions for socials not yet added."

Step 6: Public profile, themes and templates

The page at `/:username` loads the profile and renders one of several templates (Classic, Grid, Minimal, Cover, Editorial, Search & solve, Profile card) in one of several themes. Add a live phone preview in the dashboard.

Prompt: "Create a public profile page with selectable templates and themes (Light, Sage, Blush, Midnight, Auto dark). Add a sticky phone preview in the dashboard with a toolbar: Open, Copy, Share, QR, Stats."

Step 7: Onboarding

After the first log-in, show a 5-step wizard: personal or business → business category and optional WhatsApp → name, bio, photo → socials → template. Save `onboarded_at` so it shows only once.

Prompt: "Add a step-by-step onboarding modal on first login with progress bar, back/next, skip, and save everything at the end."

Step 8: Animation

```
npm install gsap three
```

Use Framer Motion for hover/press springs, staggered reveals and page transitions; GSAP ScrollTrigger for the scroll-clip panel; three.js (loaded lazily) for the 3D tree. Respect "reduce motion" settings everywhere.

Prompt: "Animate the hero headline word by word with a rotating highlighted word, add a looping 3-step story inside the hero phone, and a section whose rounded panel opens to full width on scroll using GSAP ScrollTrigger."

Step 9: Your own analytics

An `events` table gets one row per view or click: referring site, device type, country (from Vercel's `x-vercel-ip-country` header) and a random visitor id from a cookie, so refreshes and repeat visits count once. Don't store IP addresses. Skip bots and the owner's own visits. Chart it in the dashboard.

Prompt: "Build in-house analytics: record views and clicks with referrer, device, country and an anonymous visitor cookie; show views, unique visitors, clicks, CTR, a daily chart, top links, countries, sources and devices."

Step 10: Email

Sign up at Resend, verify your domain, and send emails with one `fetch` call. Store only a hash of each one-time token, with an expiry. In development, print emails to the terminal instead.

Prompt: "Add email verification and forgot/reset password using Resend. Tokens are single-use, hashed and expire. A reset signs out other sessions."

Step 11: Paid features and Paystack

List the paid features in one file (`server/features.js`). Store what each user has bought in a `user_features` table with an `expires_at`, and check it on the server before allowing a paid action (4th link, premium template, founder's note...). Prices are a monthly amount per feature plus discounts for 3, 6 and 12 months. The founder edits them in the dashboard (an `app_settings` table), with `.env` as the fallback. Payment flow:

1. "Unlock" on a feature → server calls Paystack `transaction/initialize` with the price and puts the feature and months in the metadata → user goes to Paystack checkout.
2. Paystack sends them back to `/billing/callback?reference=...` → server calls `transaction/verify` → if paid in full, unlock the feature (or add the months to what's left).
3. Paystack also calls your **webhook** (`/api/billing/webhook`, signed with your secret key) so the feature still unlocks if the customer closes the tab.

In the dashboard, show a checklist of every feature: ticked with an expiry date when unlocked, struck through with a price and an Unlock button when not.

Prompt: "Add pay-per-feature: each feature can be bought for 1, 3, 6 or 12 months via Paystack, verified on the server and by signed webhook, stored with an expiry. Show a feature checklist in the dashboard (locked ones struck through) and a pricing editor in the founder dashboard."

Step 12: Founder dashboard on a subdomain

One endpoint returns whole-site numbers, allowed only for `OWNER_EMAIL` after that email is verified. The frontend checks the hostname: on `admin.linqsafe.com` it shows only the founder dashboard and log-in. Setting `ADMIN_HOST` makes the server refuse founder requests from any other host.

Prompt: "Create a founder dashboard (users, signups per day, paying users, revenue, business counts, traffic, countries, categories, latest signups with last login, contact messages) served on the admin subdomain and locked to my verified email."

Step 13: Make it deployable to Vercel

Split the server into `server/app.js` (exports the app) and `server/index.js` (starts it locally). Add `api/index.js` that exports the app for Vercel, and a `vercel.json` that sends `/api/*` to it and everything else to `index.html`. Add `@vercel/analytics` for site-wide traffic.

Optional: Docker

Only needed if you'll host on a Docker platform (Render, Railway, Fly, a VPS) or want everything in containers. Install Docker Desktop, then:

- **Dockerfile**, two stages: (1) `node:22-alpine` installs everything and runs `npm run build`; (2) a slim image with production dependencies, `server/` and `dist/`, running as the non-root `node` user on port 3001 with a health check on `/api/health`.
- **.dockerignore**: keep `.env`, `node_modules`, docs and tests out of the image. Secrets are passed in at run time.
- **docker-compose.yml**: a `db` service (MySQL 8.4 with a volume and a health check) and an `app` service that waits for it and connects to host `db`.

```
docker compose up --build # app + MySQL on http://localhost:3001
docker build -t linqsafe --build-arg APP_STAGE=prod .
docker run -p 3001:3001 --env-file .env linqsafe # app only, hosted DB
```

Prompt: "Add a two-stage Dockerfile for the Vite frontend and Express API (non-root user, health check), a `.dockerignore`, and a `docker-compose.yml` with MySQL 8.4 and the app."

Step 14: Polish

Disclaimers where info becomes public, an onboarding-friendly empty state, a stage badge (LOCAL/DEV), a footer, a pricing page, and accessibility (labels, focus rings, keyboard reordering, alt text).

5. Going live: local, dev and prod

Three stages, each with its own database and settings, so testing never touches real users or money.

Stage	Address	Database	Settings	Branch	Paystack
Local	localhost:5173	lingsafe_local on your computer	.env.local	usually dev	test
Dev	dev.lingsafe.com	cPanel MySQL ... _lingsafe_dev	.env on the server	dev	test
Prod	lingsafe.com, admin.lingsafe.com	cPanel MySQL ... _lingsafe	.env on the server	prod	live

The server reads `.env.local` first and `.env` second (first value wins), so your computer never uses a live database by accident. Work on `dev`, deploy it to `dev.lingsafe.com`, check it, then `git merge dev` into `prod` and deploy that.

Hosting on Namecheap cPanel

Namecheap's MySQL only accepts connections from its own server, so the app runs right next to it as a cPanel **Node.js App** (Phusion Passenger):

1. Point the domain at the hosting: nameservers **Namecheap Web Hosting DNS**. Create subdomains `dev` and `admin` in cPanel → Domains.
2. cPanel → MySQL Databases: one database and user for dev, another for prod.
3. On your computer: `npm run package:dev` (dev branch) or `npm run package` (prod branch). This builds the site and zips only what the server needs: no settings files.
4. File Manager: create the app folder outside `public_html`, upload and extract the zip, and create a `.env` with that stage's database, `JWT_SECRET`, `APP_URL`, Paystack keys and `NODE_ENV=production`.
5. Setup Node.js App → Create Application: root = the folder, URL = the (sub)domain, startup file `app.cjs` → Run NPM Install → Restart.
6. Check `/api/health`; turn on AutoSSL; set the Paystack webhook (test mode → dev, live mode → prod).

Updating: build the zip, upload and extract over the folder (the `.env` stays), Restart. Database changes apply automatically on restart. Full details, backups and troubleshooting are in `docs/DEPLOY.md`.

Prompt: "Set up local, dev and prod stages with separate databases and settings files, a startup file for cPanel's Node.js App, and a script that builds a deploy zip per stage."

6. Launch checklist

- A long random `JWT_SECRET` per stage (`openssl rand -hex 32`).
- Owner account created with `OWNER_EMAIL` and the email verified.
- One full Paystack test payment on dev: checkout → callback → feature unlocked → webhook received.

- Sign-up, verify email, reset password tested with a real inbox.
- Privacy policy mentions the analytics cookie, hosted images and Paystack; add a cookie notice if you have EU/UK visitors.
- Database backups switched on at your MySQL host.
- Test on a real phone: onboarding, preview sheet, each template.